

Data Science Life Cycle

Structured process to extract insights from raw data. Used in industries like healthcare, finance, and marketing. **Data Collection:** Gather raw data. **Data Processing:** Clean and organize data. **Data Analysis:** Apply statistical or machine learning models. **Interpretation:** Derive insights and conclusions.

Artificial Intelligence (AI)

Machines programmed to learn, reason, and act like humans. Automate tasks, improve efficiency, solve complex problems. Used in Robotics, chatbots, self-driving cars.

Machine Learning (ML)

Train algorithms to find patterns in data without explicit programming. Build predictive models when rules are hard to define. Recommendation systems, fraud detection.

Python

High-level, interpreted programming language. Easy syntax, vast libraries, versatile for data science. Web development, data analysis, AI, automation.

Pandas

Python library for data manipulation and analysis. Simplifies data cleaning, exploration, and analysis. **Series:** 1D labeled array (single column). **DataFrame:** 2D labeled data (rows and columns). **Index:** Row/column labels for easy data access. **Features:** Data alignment, reshaping, handling missing data. `elections = pd.read_csv("elections.csv")` **Get Data Shape** Get number of rows and columns: `df.shape` **Data Slicing Techniques** **Using []:** Access a Single Column: `df['column_name']` Access Multiple Columns: `df[['col1', 'col2']]` Range of Rows: `df[0:3]` #First 3 rows Range of Rows and Columns: `df.loc[:, 'col1':'col3']` # All rows, columns 'col1' to 'col3' **Bracket [] Indexing** Select columns or filter data: `df['col_name']` # Select a column `df[df['col_name'] > value]` #Filters rows where 'col_name' > value **Boolean Indexing:** `df[df['Year'] == 1828]['Candidate']` #Filter rows where Year = 1828, then show Candidate **Loc (Label-based) loc[]:** `df.loc['row_label', 'col_label']` #Specific row/column by labels `df.loc[df['Party'] == 'Democratic']` #Filter rows where Party = Democratic `df.loc[df['Result'] == 'win', ['Year', 'Popular vote']]` #Filter & show specific columns `df.loc[0:2, 'col1':'col3']` # Rows 0-2, columns from 'col1' to 'col3' Filter elections with percentage > 55%: `df.loc[df['%'] > 55]`

Iloc (Index-based) iloc[] : `df.iloc[0:5, 2:4]` #Rows 0-4, Columns 2-3 `df.iloc[0, 1]` # First row, second column Display first 5 rows: `df.iloc[:5]` Select data from second to fourth row, first three columns: `df.iloc[1:4, :3]` Extract 'Popular vote' and 'Result' columns for first 10 rows: `df.iloc[:, 10, [3, 4]]` **Combining Conditions** Rows where result is 'loss' and percentage < 50%: `df.loc[(df['Result'] == 'loss') & (df['%'] < 50)]` Candidates from 'Democratic-Republican' party who won: `df.loc[(df['Party'] == 'Democratic-Republican') & (df['Result'] == 'win'), 'Candidate']` **Bonus Challenge** Create 'Vote Margin' column and display top 5 candidates: `df['Vote Margin'] = df['Popular vote'] - df.groupby('Year')['Popular vote'].transform('mean')` `df.nlargest(5, 'Vote Margin')['Candidate', 'Vote Margin']`

Grouping and Aggregation **Grouping:** Organize data into groups based on column values. **Aggregation:** Apply functions like mean, sum, or count to grouped data. `df.groupby('column_name').agg_function()` **Mean:** `df.groupby('column_name').mean()` **Sum:** `df.groupby('column_name').sum()` **Count:** `df.groupby('column_name').count()` **Q: Which of the following pandas statements returns a DataFrame of the first 3 baby names with Count > 250:** `babynames.loc[babynames["Count"] > 250, ["Name", "Count"]].iloc[0:2, :]` **Numpy Quick Reference** **Array Creation:** `import numpy as np` `arr = np.array([1, 2, 3])` **Common Functions:** `np.mean(arr)`, `np.sum(arr)`, `np.max(arr)`, `np.min(arr)` **Lists vs. NumPy Arrays** **Data Type: Lists:** Can store mixed data types. **NumPy Arrays:** Store homogeneous data types. **Performance: Lists:** Slower due to dynamic typing. **NumPy Arrays:** Faster, optimized for numerical computations. **Memory Usage: Lists:** Consume more memory due to flexibility. **NumPy Arrays:** More memory-efficient for numerical data. **Functionality: Lists:** Basic operations (append, remove). **NumPy Arrays:** Advanced operations (math functions, vectorized operations). **Indexing: Lists:** Simple indexing. **NumPy Arrays:** Advanced slicing and broadcasting.

Libraries: Lists: Built-in Python. **NumPy Arrays:** Require `import numpy as np`. **Boolean Arrays:** You create a Boolean array by applying a condition to a NumPy array or Pandas DataFrame. `import numpy as np` `arr = np.array([1, 2, 3, 4, 5])` `bool_arr = arr > 3` # [False, False, False, True, True] Using Boolean Arrays for Filtering: Use a Boolean array to select elements that meet a condition. `result = arr[bool_arr]` # [4, 5] **Boolean Arrays in Pandas:** You can filter rows in a DataFrame based on conditions. `import pandas as pd` `df = pd.DataFrame({'A': [10, 20, 30], 'B': [40, 50, 60]})` `filtered_df = df[df['A'] > 15]` # Rows where column 'A' > 15 **Combining Multiple Conditions:** Use logical operators like & (AND) or | (OR). `filtered_df = df[(df['A'] > 15) & (df['B'] < 55)]` # Multiple conditions Ensure that the Boolean array has the same length as the array or DataFrame being filtered. Use parentheses () when combining multiple conditions to avoid errors. **Creating a Pandas Series** **From a List:** `series = pd.Series([10, 20, 30])` #Default index: 0, 1, 2 **From a Dictionary:** `series = pd.Series({'a': 100, 'b': 200})` #Keys become the index **From a Numpy Array:** `series = pd.Series(np.array([1.5, 2.5]), index=['x', 'y'])` #Custom index: x, y **Pivot Tables** Reshapes data, aggregating based on values. Simplifies complex data summaries. `df.pivot_table(values, index, columns)` `df.pivot_table(values='value', index='index_col', columns='col', aggfunc='mean')` Pivot tables summarize data by reshaping it. Use `pivot_table()` to create them. Example: Pivot with Party and Year `elections.pivot_table(values='%', index='Party', columns='Year', aggfunc='mean')` **Using a Lambda Function** A lambda function is an anonymous function useful for custom computations. Example: Sort and Group to Find the First Row `elections_sorted_by_percent = elections.sort_values("", ascending=False)` `elections_sorted_by_percent.groupby("Party").agg(lambda x: x.iloc[0])` **Using idxmax** The `idxmax` function finds the index of the maximum value in a Series. Example: Find the Best Percentage for Each Party `best_per_party = elections.loc[elections.groupby("Party")["%"].idxmax()]` **Using drop_duplicates** This method removes duplicate rows based on specified columns. Example: Find the Best Percentage per Party **Using drop_duplicates** `best_per_party2 = elections.sort_values("").drop_duplicates(["Party"], keep="last")` This sorts the DataFrame by %, removes duplicate rows based on Party, and keeps the last row (highest percentage for each party).

.isin() The `.isin()` method is used to filter rows based on whether the values in a column exist in a specified list or other iterable (like a set or another Series). `df[column].isin(values)` # Filter rows where Name is either 'Alice' or 'Bob' `df[df['Name'].isin(['Alice', 'Bob'])]` **.str.startswith()** The `.str.startswith()` method is used to check if each string in a Series starts with a given substring. `df[column].str.startswith(substring)` # Filter rows where the Name starts with 'A' `df[df['Name'].str.startswith('A')]` **.groupby().filter()** The `.groupby().filter()` method is used to filter data based on the group-level condition. It works by first applying a grouping operation and then filtering out entire groups based on a condition applied to each group. `df.groupby('column').filter(lambda x: condition)` **column:** The column to group by. **condition:** A lambda function (or a condition function) that will be applied to each group. The group will be kept if the condition is True. # Keep only groups where the sum of 'Value' is greater than 50 `df.groupby('Category').filter(lambda x: x['Value'].sum() > 50)` **1. Custom Sorts** You can sort a DataFrame based on specific criteria using `sort_values`. Custom sorts allow you to control the order using either an external mapping or the logic inside your code.

Example: Sorting by a specific column `elections_sorted_by_percent = elections.sort_values("", ascending=False)` This sorts the DataFrame in descending order based on the % column. **2. Adding, Modifying, and Removing Columns** You can add new columns, update existing ones, or delete them. Add or Modify a Column: `elections['Vote Margin'] = elections['Popular vote'] - elections['Electoral vote']` Remove a Column: `elections = elections.drop(columns=['Vote Margin'])` **3. groupby().agg** The `agg()` function is used to perform aggregate computations on groups. You can pass functions like `sum`, `mean`, `max`, or even custom lambda functions. Example: Group by Party and Get the Maximum Percentage `elections.groupby("Party").agg(lambda x: x.iloc[0])` # First row of each group **4. Sorting by String Length** You can sort rows by the length of a column's string values. Example: `elections['Name Length'] = elections['Candidate'].str.len()` `elections_sorted = elections.sort_values('Name Length')` **5. groupby().size** The `.size()` method counts the number of rows in each group. Example: `elections.groupby('Party').size()` This will return the number of elections for each party. **6. groupby().filter** Filters groups based on a condition applied to each group. Example: Filter Parties with More Than 2 Elections `elections_filtered = elections.groupby("Party").filter(lambda x: len(x) > 2)`

Difference Between .loc and .iloc: **.loc:** Label-based indexing (uses row/column names). **.iloc:** Integer-based indexing (uses row/column positions). **What is the pandas library?** A Python library for data manipulation and analysis with structures like DataFrames and Series. **Significance of pandas:** Simplifies handling, cleaning, and analyzing structured data efficiently. **What is the NumPy library?** A library for numerical computing, supporting arrays, matrices, and mathematical functions. **Significance of NumPy:** Enables fast array operations and mathematical computations. It's the foundation for many data science libraries like pandas and scikit-learn. **Machine Learning:** **What is a model in machine learning?** A mathematical representation that learns patterns from data to make predictions or decisions. **10. Why are models important?** They generalize patterns from data to solve real-world problems, such as regression or classification. **Key Questions and Answers:** **What is the significance of pandas in Python?** It provides tools for efficient data manipulation and analysis of structured data. **What is a DataFrame in pandas?** A 2-dimensional, tabular data structure with labeled rows and columns. **What are the key data structures in pandas?** **Series:** 1D data structure. **DataFrame:** 2D data structure. **More on .loc and .iloc:** **How is .iloc different from .loc?** **.iloc:** Integer-based indexing. **.loc:** Label-based indexing. **What does df.iloc[:,2] return?** Every other row of the DataFrame, starting from the first. **Grouping and Aggregation:** **What does df.groupby('col') do?** Groups the DataFrame by unique values in the specified column. **How do you apply aggregation after grouping?** Use `.agg()` or functions like `.sum()`, `.mean()` on the grouped object. **What does df.groupby('col').size() return?** The number of rows in each group. **What does df.groupby('col').sum() return?** The sum of values in each group for numerical columns. **Can you group by multiple columns?** Yes, use `df.groupby(['col1', 'col2'])`. **Reading and Writing Data:** **How do you read a CSV file in pandas?** Use `pd.read_csv('file.csv')`. **How do you write a DataFrame to a CSV file?** Use `df.to_csv('file.csv')`. **Handling Missing Values:** **How do you check for missing values in a DataFrame?** Use `df.isnull()` or `df.isnull().sum()`. **How do you drop rows with missing values?** Use `df.dropna()`. **How do you replace missing values?** Use `df.fillna(value)`. **Data Sorting:** **How do you sort a DataFrame by a column?** Use `df.sort_values('column')`.

Q Suppose we have a dataset of one feature. We collect n observations of the form (xi, yi). In other words, we record one scalar feature xi and one scalar target variable yi for the i-th datapoint. We model each observed yi using a model with one scalar parameter, 0.

$$\hat{y_i} = f_{\theta}(x_i) = \theta \cdot \ln(|x_i| + 1)$$

Determine the optimal value $\hat{\theta}$ that minimizes the mean squared error (MSE) of this model on the dataset in terms of xi, yi. You may assume that the critical point of this loss function is guaranteed to be a minimum.

Step 1: Simplify the derivative

$$\frac{\partial L(\theta)}{\partial \theta} = \frac{1}{n} \sum_{i=1}^n 2(y_i - \theta \ln(|x_i| + 1)) \cdot (-\ln(|x_i| + 1))$$

Simplifying further:

$$\frac{\partial L(\theta)}{\partial \theta} = -\frac{2}{n} \sum_{i=1}^n (y_i - \theta \ln(|x_i| + 1)) \ln(|x_i| + 1)$$

Step 2: Set the derivative to 0

To find the minimum of L(θ), we set the derivative to 0:

$$0 = -\frac{2}{n} \sum_{i=1}^n (y_i - \theta \ln(|x_i| + 1)) \ln(|x_i| + 1)$$

C canceling the -2/n term:

$$0 = \sum_{i=1}^n (y_i - \theta \ln(|x_i| + 1)) \ln(|x_i| + 1)$$

Step 3: Solve for θ

Expand the summation:

$$0 = \sum_{i=1}^n y_i \ln(|x_i| + 1) - \theta \sum_{i=1}^n \ln(|x_i| + 1)^2$$

Rearrange:

$$\sum_{i=1}^n y_i \ln(|x_i| + 1) = \theta \sum_{i=1}^n \ln(|x_i| + 1)^2$$

Solve for θ:

$$\hat{\theta} = \frac{\sum_{i=1}^n y_i \ln(|x_i| + 1)}{\sum_{i=1}^n \ln(|x_i| + 1)^2}$$

Why Not Use Residual Error Directly as a Loss Function?
Issue: Positive and negative residuals can cancel each other out.

Example: Large errors in opposite directions could sum to zero, misleadingly indicating perfect performance.

Result: The model might seem accurate even when predictions are far off.

When to Use MSE or MAE

Mean Squared Error (MSE)

Key Feature: Penalizes large errors heavily (squares the error).

Differentiable: Suitable for optimization (e.g., gradient descent).

Best for:

Regression tasks where large errors matter more.

Model training as a loss function.

Mean Absolute Error (MAE)

Key Feature: Treats all errors equally.

Robust to Outliers: Less affected by extreme values.

Best for:

Minimizing average error magnitude.

Datasets with significant outliers.

Choosing the Right Metric:

Outliers present? Use MAE.

Need differentiability? Use MSE.

Interpretability? MAE is more intuitive (same units as the target).

Summary:

MSE: Penalizes large errors, differentiable.

MAE: Minimizes average error, robust to outliers.

Simple Linear Regression:

Equation: y=mx+b

y (Dependent Variable): The outcome you want to predict.

x (Independent Variable): The predictor or input variable.

m: Slope (rate of change).

b: Intercept (value of y when x = 0).

Steps: Calculate the slope (m) and intercept (b) using data. Substitute x values into the equation to predict y.

- Identifies and quantifies relationships between two variables.

- Used in various fields like finance, biology, and economics for trend analysis and forecasting.

Models the relationship between two variables with a straight line.

To predict one variable based on another.

Used for trend analysis and forecasting.

Error Metrics:

Mean Squared Error (MSE):

$$MSE = \frac{1}{n} \sum (y_i - \hat{y}_i)^2$$

Measures average squared differences between actual and predicted values.

Sensitive to outliers due to squaring errors.

Mean Absolute Error (MAE):

$$MAE = \frac{1}{n} \sum |y_i - \hat{y}_i|$$

Measures average absolute differences between actual and predicted values.

Less sensitive to outliers compared to MSE.

What is Simple Linear Regression?

A method to model the relationship between one independent variable and one dependent variable.

What is the goal of Simple Linear Regression?

To minimize the error between predicted and actual values.

What is Mean Squared Error (MSE)?

The average squared difference between actual and predicted values.

What is Mean Absolute Error (MAE)?

The average absolute difference between actual and predicted values.

How do you fit a linear regression model in Python?

Use libraries like **scikit-learn**:

from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X_train, y_train)

Powerful methods for data manipulation in Pandas:

Custom sorts: Organize data by any criteria.

Adding, modifying, removing columns: Reshape your DataFrame as needed.

Groupby operations: Summarize and filter groups effectively.

Pivot tables: Reorganize data for aggregated views.

Lambda functions: Apply custom logic.

idxmax and drop_duplicates: Efficiently find key values.

Loss Functions:

Why is the Mean Squared Loss function used?

Penalizes larger errors more heavily, making it sensitive to outliers; ideal for regression tasks.

Why is the Mean Absolute Loss function used?

Treats all errors equally and is more robust to outliers compared to Mean Squared Loss.

Grouping in Pandas:

Why is grouping used in pandas?

Aggregates data (e.g., sum, mean) based on unique values in a column, simplifying subset analysis.

Three Reasons for Building Models:

1. Explain Complex Phenomena:

Understand relationships between variables.

Example: How do parents' heights relate to children's heights?

Focus: Simple, interpretable models.

2. Make Accurate Predictions:

Predict outcomes for unseen data.

Example: Is an email spam?

Focus: Accuracy over interpretability (e.g., black-box models in deep learning).

3. Infer Causation:

Determine if one factor causes another.

Example: Does smoking cause lung cancer?

Focus: Advanced statistical methods (future courses).

Balancing **interpretability** and **accuracy** is key in most cases.

Goals of Visualization:

To help understand your data/results.

To communicate results/conclusions to others.

Visualization Distribution:

The set of values that a variable can take

The frequency with which each value occurs

Sum of frequencies = 100.

Variable Types in Data Analysis:

Quantitative: Continuous (e.g., height), Discrete (e.g., yes/no, 0-8).

Qualitative: Ordinal (e.g, ordered/ranked data), and Nominal.

Plotting Notes:

If the variable type is qualitative, use a bar plot (NOT for quantitative).

We mainly use two libraries for generating plots: Matplotlib and Seaborn.

If we use a bar plot for quantitative, it will lead to multiple bars for continuous data which will produce inaccurate results.

Use a box plot, violin plot, or histogram if the variable type is quantitative.

For a quantitative variable, quartiles are used.

Multiple Regression:

Extension of linear regression for one dependent variable and multiple independent variables. Finds best-fitting line/hyperplane to minimize error. Assumes a linear relationship between dependent and independent variables.

$$y = \beta_0 + \beta_1 \beta_1 + \beta_2 \beta_2 + \cdots + \beta_n \beta_n + \epsilon$$

Where:

y : Dependent variable (response).

x1,x2,...,xn : Independent variables (predictors).

β0 : Intercept. (Value of y when all xi = 0)

β1,β2,...,βn : Coefficients of the independent variables, representing the change in y for a one-unit change in xi,

holding other variables constant.

ε : Error term (accounts for variability not explained by the model).

Logistic Regression:

Used for binary classification (outcome: 0 or 1).

Predicts the probability of the positive class (1) using the logistic function (sigmoid).

Outputs values between 0 and 1, representing the probability.

Models the log-odds of the outcome.

$$p(y = 1 | x_1, x_2, \dots, x_n) = \frac{1}{1 + e^{- (\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n)}}$$

broken down formula:

$$p(y = 1 | x_1, x_2, \dots, x_n) = \frac{1}{1 + e^{\kappa}}$$

$$\kappa = - (\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n)$$

$p(y = 1 | x_1, x_2, \dots, x_n)$: Probability of the outcome

being 1 (positive class).

$x_1, x_2, \dots, x_n$: Independent variables (predictors).

$\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n$: Coefficients (intercept and weights).

e: Euler's number (base of the natural logarithm).